**String methods**:

- `indexOf()`
- `startsWith()`
- `endsWith()`
- `contains()`
- `substring()`
- `charAt()`
- `length()`
- `equals()`
- `equalsIgnoreCase()`
- `compareTo()`
- `compareToIgnoreCase()`

The best way to reinforce them is with **real-world business scenarios** instead of direct method-based questions.

Here are **15 real-time problem statements** suitable for Java students.

---

# 1. Email Validation System

### Problem Statement

A company allows only Gmail accounts.

Accept an email from the user and verify:

- It contains `@`
- It ends with `.com`
- It contains `"gmail"`

Example

Input:
rahul@gmail.com

Output:
Valid Gmail Account

Methods Used

contains()

endsWith()
indexOf()

---

# 2. Employee ID Verification

Employee IDs always start with

EMP

Check whether the entered ID is valid.

Example

Input:
EMP1025

Output:
Valid Employee ID

Methods

startsWith()
length()

---

# 3. Password Strength Checker

Rules

- Minimum 8 characters
- Must contain "@"

Example

Input:
Java@123

Output:
Strong Password

Methods

length()
contains()

---

# 4. Website Domain Checker

Accept a website URL.

Tell whether it belongs to:

.com
.org
.in
.edu

Example

Input:
royaltechnosoft.com

Output:
Commercial Website

Methods

endsWith()

---

# 5. Student Roll Number

Roll numbers follow:

RTS2026001

Extract only

2026001

Methods

substring()

---

# 6. Username Initial Generator

Input

Rahul

Output

First Letter : R

Methods

charAt(0)

---

# 7. Product Search

Products

Laptop
Mouse
Keyboard
Monitor

User searches

mouse

Search without considering uppercase/lowercase.

Methods

equalsIgnoreCase()

---

# 8. Login Authentication

Stored Username

admin

Stored Password

Java123

Compare entered credentials.

Methods

equals()
equalsIgnoreCase()

---

# 9. File Extension Checker

Input

Report.pdf

Output

PDF File

Methods

endsWith()

---

# 10. Mobile Number Validation

Rules

- Exactly 10 digits
- Starts with

9
8
7
6

Methods

length()
startsWith()
charAt()

---

# 11. Search Word in Sentence

Input

Java Programming Language

Search

Programming

Output

Word Found

Methods

contains()
indexOf()

---

# 12. Name Formatter

Input

Rahul Kirpekar

Output

First Name : Rahul
Last Name : Kirpekar

Methods

indexOf()
substring()

---

# 13. Lexicographical Name Sorting

Compare

Rahul
Raj

Tell which comes first alphabetically.

Methods

compareTo()

---

# 14. Search Character Position

Input

Royal Technosoft

Find

T

Output

Found at index 6

Methods

indexOf()

---

# 15. Company Email Verification (Combined Problem)

A company accepts emails only if:

- Starts with employee ID `emp`
- Contains `@royaltechnosoft`
- Ends with `.com`
- Username before `@` should have at least 5 characters

Example

Input:
emprahul@royaltechnosoft.com

Output:
Valid Company Email

Methods

startsWith()
contains()
endsWith()
indexOf()
substring()
length()

---

# ⭐ Bonus Mini Project (Uses Almost Every Method)

## Royal Technosoft Student Registration Validation

### Problem Statement

Create a Java program that validates a student's registration details.

Input:

Name
Email
Password
Course Code

Validation Rules:

- Name should not be empty.
- Name must start with a capital letter.
- Email must contain `@`.
- Email must end with `.com`.
- Password must be at least 8 characters long.
- Password must contain `@`.
- Course code should start with `JAVA`.
- Extract the batch number from the course code.
- Compare the entered course code with `JAVA101` (ignore case).
- Compare the student's name with `"Admin"` (case-sensitive).
- Print the first character of the student's name.
- Display the position of `@` in the email.

## Methods Covered

| Method | Used |
|---|---|
| `length()` | ✔ |
| `charAt()` | ✔ |
| `substring()` | ✔ |
| `contains()` | ✔ |
| `indexOf()` | ✔ |
| `startsWith()` | ✔ |
| `endsWith()` | ✔ |
| `equals()` | ✔ |
| `equalsIgnoreCase()` | ✔ |
| `compareTo()` | ✔ |

`compareToIgnoreCase()` ✔

This final project is ideal as a **lab assessment or practical exam** because it naturally requires students to apply nearly all the string methods you've taught in one realistic scenario.